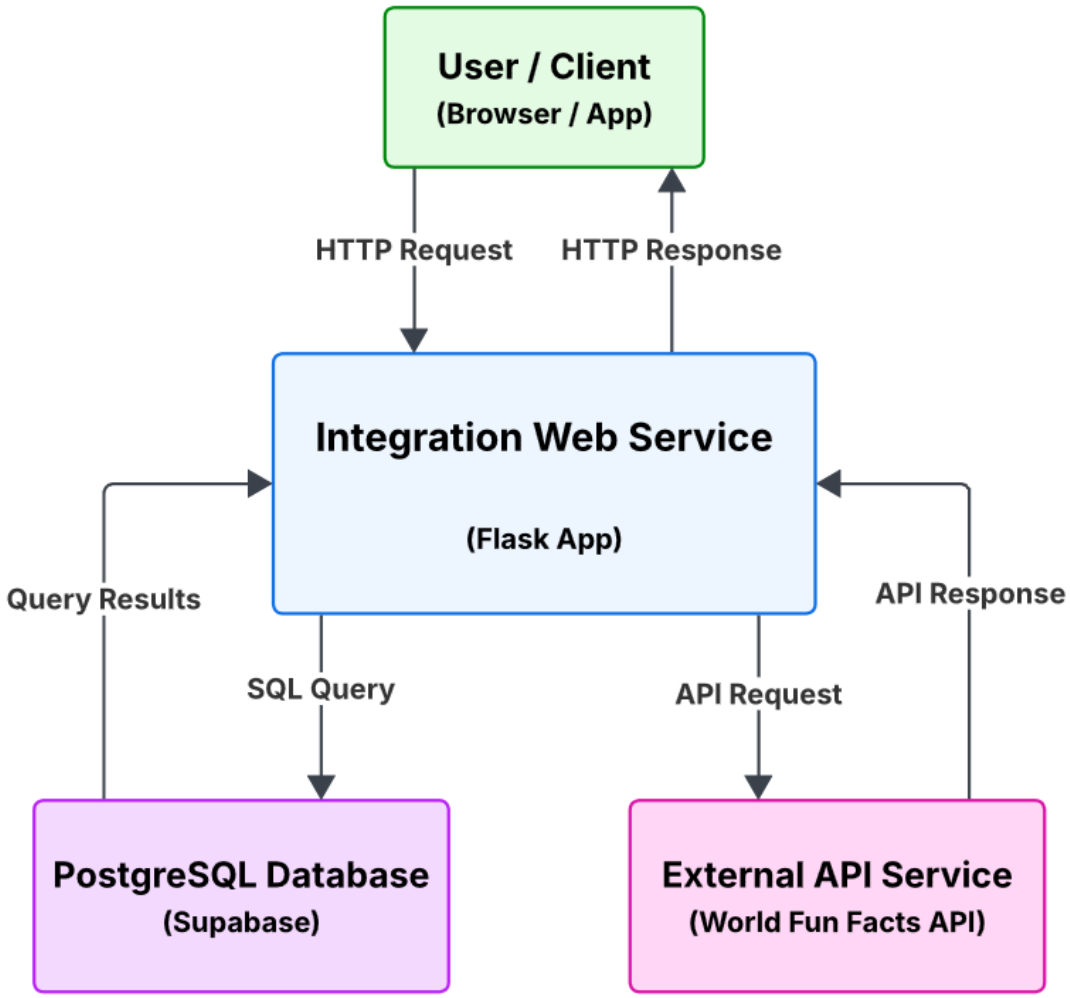
Architecture Notes

Group Number: 22

Members: Maria O' Flynn (124423752) | Anastasija Smentanina (119389906)

Context Diagram



Integration Points

Our flask application integrates two external services:

- A PostgreSQl database (which is hosted on Supabase)
- Our World Fun Facts API (found and hosted via RapidAPI)

The database stores any retrieved facts in a *facts* table containing data fields such as ***uuid, text and lang_code*.** The API is called upon and returns a random fact in JSON format. Our app also exposes four main endpoints:

- "/" (shows a HTML page with the latest fact gathered from the database)
- "/new-fact" (this fetches a new fact from the API, stores it in the database and redirects you back to "/")
- "/fact" (shows the JSON of the latest fact)
- "/health" (this checks the connectivity to the database and the API).

Branching Model

We used trunk-based development, which revolved around short-lived branches that merge into a single branch (https://www.atlassian.com/continuous-delivery/continuous-integration/trunk-based-development)

- Main branch – master
- Other branches – feature/dockerfile & feature/requirements

Every pull request requires at least one other person (in our case, the other person) to check the changes, and all status checks must pass before it can be merged.

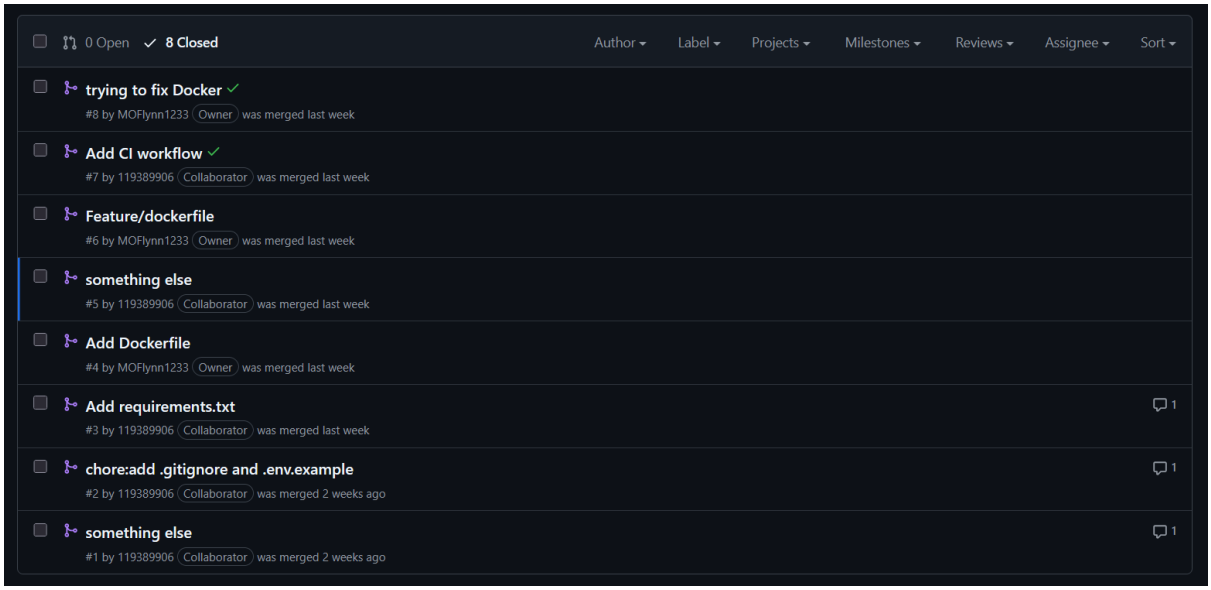


Figure 1 – Screenshot of pull requests log from GitHub

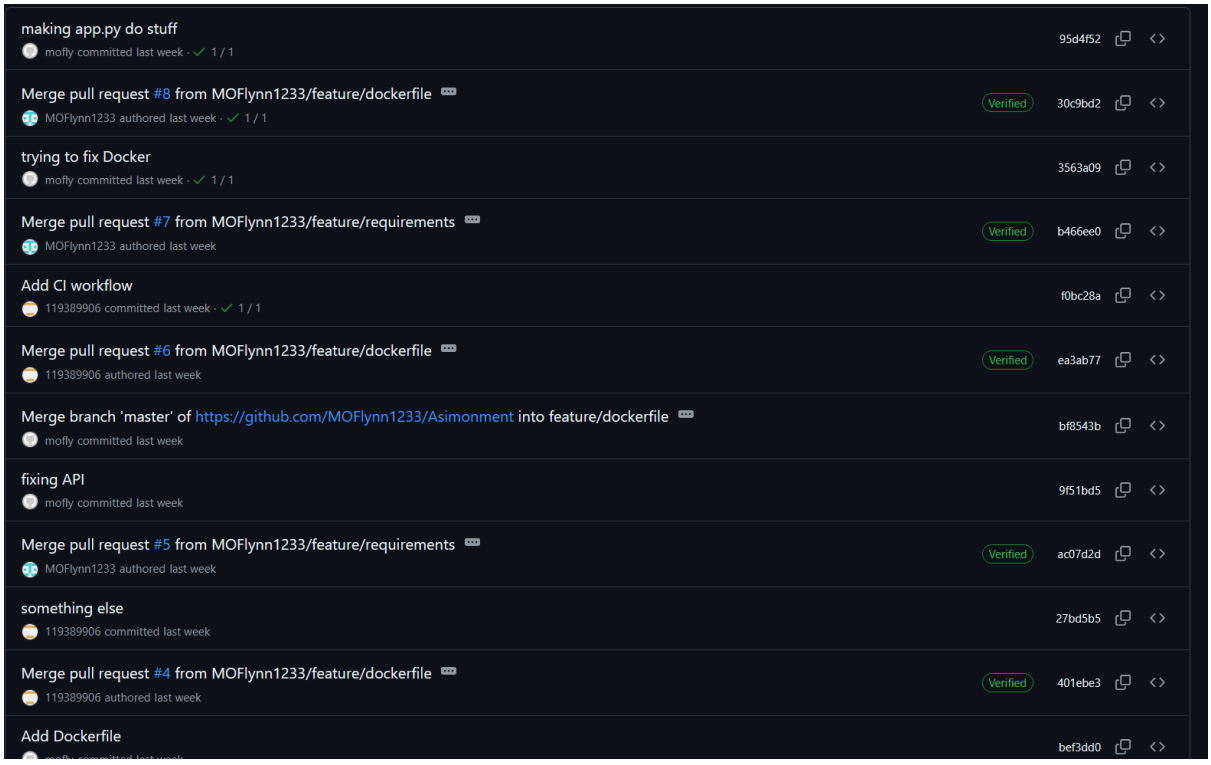


Figure 2 – Screenshot of the commits log from GitHub

CI/CD Pipeline Design

The CI/CD pipeline is implemented with GitHub Actions. It runs on every pull request and on every push to the master branch. The pipeline consists of the following stages:

- Checkout: retrieves the latest code.
- Set up Python: installs Python 3.11.
- Install dependencies: uses pip to install packages from requirements.txt (including ruff & pytest).
- Lint: runs ruff check . to enforce code style and catch common errors.
- Test: executes pytest with coverage reporting. The coverage report is printed in the workflow log.
- Build Docker image – creates a test image named test-image to verify the Dockerfile is valid.

After the image is pushed, the live deployment on Render automatically pulls the new image and redeploys the application (triggered by Render's GitHub integration).